

Q. Who uses your product?

A. We're building a better version of Azure AI Foundry that allows enterprises to host optimised AI models on-premises, delivering high accuracy, security, and performance. Several enterprises already use it. Large service providers like Infosys, Capgemini, LTI mindtree and Accenture leverage our platform to quickly create scalable, secure AI solutions for their clients. Cloud companies use our system to move beyond selling GPU hardware and offer AI-powered services like models or tokens as a service, adding more value on top of their infrastructure.

OEMs like Dell and Netweb use our technology to support multiple hardware platforms—including Intel, NVIDIA, AMD, Qualcomm, and ARM—and provide AI solutions alongside their devices, enabling them to sell outcomes rather than just specs. Think of it like selling a PC — instead of just promoting the hardware specs, they sell improved performance and productivity. Semiconductor companies also adopt our stack from day one. These are the main customer groups we serve.

Q. What are the basic needs to run your software on any hardware? Do you also offer it as a SaaS?

A. We plan to launch a SaaS version in the next 6 to 12 months. Right now, we offer on-prem licences for large companies, cloud providers, and systems integrators. As for hardware, our software can run on low-cost CPUs—like a \$200 server from AWS. That means you can run genAI applications on basic, low-spec machines, like those used for simple web apps. And it's not just limited use—we support high-performance, non-quantized LLM inference with many requests at once, even on this kind of hardware.

Q. How do you reduce hardware limits but keep performance high?

A. We're building a platform to minimise hardware dependency for genAI infrastructure—something currently nearly impossible. Systems built on NVIDIA CUDA are locked into NVIDIA hardware and even deploying across different NVIDIA chips is difficult due to varying CUDA versions. Mixing hardware from different vendors like NVIDIA, AMD, and Intel—or combining GPUs, HPUs, and CPUs—is currently unfeasible.

We're the only company addressing this with heterogeneous parallelism and clustering, plus SLO-based management, which abstracts hardware details. This means users don't have to worry about the underlying hardware. It's like how operating systems solved the early mainframe problem by providing hardware abstraction and portability. Today, genAI faces a similar issue -- models built on NVIDIA can't easily move elsewhere. We aim to solve this, enabling AI applications to run seamlessly across diverse hardware.

Q. How does your system make CPUs run as fast as GPUs?

A. We've implemented multiple optimisations to boost performance. Custom kernels improve compute, networking, memory bandwidth, and inter-chip communication, delivering 20-30% better performance. Fused kernels optimise LLM computations on CPUs, and a new CPU-specific attention mechanism, detailed in our paper, enables efficient LLM inference.

Recently, we developed GPU-free genAI models. The main GPU bottleneck is attention, which relies on heavy matrix multiplication needing massive parallelism. Building on Microsoft's BitNet, we converted parts of attention from multiplication to addition using ternary weights. We also solved CPU memory bandwidth

limits with block approximation. These advances enable fast genAI inference on CPUs without GPUs.

Q. How can we make big genAI models use less memory and run faster?

A. There are several techniques to reduce memory usage in genAI. The most common is quantization—converting model weights from FP16 to INT8 or INT4, which reduces memory size linearly. For example, a 16GB model at FP16 drops to 8GB with INT8, or 4GB with INT4. Distillation compresses a large model's capabilities into a smaller one, cutting memory further. Offloading shifts memory from VRAM to RAM or disk, easing pressure on expensive GPU memory. Pruning removes less critical weights but requires optimised runtimes. We also use ternary weights, converting matrix multiplications into additions, which further shrinks memory usage.

Beyond model weights, the KV cache is a major memory load in autoregressive LLMs. We apply KV cache compression and explore alternative architectures like SSN and Liquid models. For scaling, we favour adapters over fine-tuned model replicas—supporting multiple use cases on a single base model. These strategies help address genAI's core challenges — expensive GPU memory and limited memory bandwidth.

Q. How does your company integrate AI into its software development processes?

A. As an AI-first company, we use AI wherever it's practical and reliable. Most of us use Cursor for a small boost in speed and efficiency, and we rely on multi-layer AI agents for tasks like PR reviews, code refactoring, documentation, and translation—helping us work across multiple programming languages. Our entire SDLC is AI-driven, supported by various tools, and recently we've added